23CSE455 Design Patterns
Case Study Report

Team A2

Case Study: MBTA API – Strategy pattern discovery, evolution, and illustrative cross-language comparison
Application: MBTA API repository, `https://github.com/mbta/api`
Date: 2026-10-15
Submission Repository: `https://github.com/aswathymohan-amrita/23CSE455_DP_CaseStudy`

Team Members

Roll Number	Student Name	Batch
AM.SC.U4CSE23201	Aadhav Gugan DU	A
AM.SC.U4CSE23220	G Yaswanth Ram Saran	A
AM.SC.U4CSE23227	KB Sai Prasad Reddy	A

1 Application and Architecture

The application studied is the MBTA API, an Elixir/Phoenix application available at `https://github.com/mbta/api`. The source was inspected from a downloaded archive. Its exact baseline Git commit could not be independently verified.

The repository specifies Elixir 1.19.5-otp-28 and Erlang/OTP 28.5.0.3 in its `.tool-versions` file.

The analysis focuses on two candidate Strategy pattern collaborations:

1. Rate Limiter Backend Selection.
2. State Data Parsing.

2 Part 1: Pattern Discovery

The candidates were identified by examining common behaviours, implementations, configuration, and delegation calls in the source code.

2.1 Rate Limiter Backend Selection

The common behaviour is `ApiWeb.RateLimiter.Limiter`. The concrete backends are `ApiWeb.RateLimiter.ETS` and `ApiWeb.RateLimiter.Memcache`. The `ApiWeb.RateLimiter` module acts as the context by selecting a configured backend and delegating operations.

This is a Strategy pattern candidate because the application can use interchangeable implementations through a shared contract.

2.2 State Data Parsing

The parsing contract is `Parse.parse/1`. The parser implementations include `Parse.Alerts`, `Parse.Line`, and `Parse.Routes`. The `State.Server` module obtains a configured parser and delegates parsing

to it.

This is a Strategy pattern candidate because parsing behavior is delegated to a selected parser implementation.

3 Part 2: Evolution and Refactoring

Three changes were prepared in the evolved source copy.

3.1 Change 1: Rate Limiter Test Coverage

Tests were added for clearing rate-limiter state and keeping different users' counters independent.

Observed result:

- Command: `mix test apps/api_web/test/api_web/rate_limiter_test.exs`
- 8 tests, 0 failures.
- Memcache and integration tests were excluded.

The Strategy pattern participants remained unchanged.

3.2 Change 2: ETS Rate Limiter Tests

Targeted tests covered requests within and over the limit, independent keys, clearing, and concurrent requests.

Observed result:

- Command: `mix test apps/api_web/test/api_web/rate_limiter/ets_test.exs`
- 5 tests, 0 failures.
- Memcache and integration tests were excluded.

The Strategy pattern remained unchanged. The recorded diff is descriptive because the original source archive did not contain the added test file.

3.3 Change 3: Parser Exception Handling

The `parse_new_state/3` function was updated to handle parser exceptions separately and avoid publishing a successful update event when parsing fails.

The targeted test command was:

```
mix test --no-compile apps/state/test/state/server_test.exs
--name-pattern "invalid parsed state does not emit a successful update event"
```

Observed result: 1 test passed and 18 were excluded. Because the command used `-no-compile`, it does not prove that the latest source compiles cleanly. A separate compile/test attempt encountered a warning treated as an error in `apps/state/lib/state/shape.ex`.

The State Data Parsing pattern remained in place; the change improved exception handling.

4 Part 3: Cross-Language Comparison

Illustrative examples were created in Java, Python, JavaScript, and C++ for both pattern candidates.

Language	Rate Limiter Example	State Parser Example
Python	Pass; delegation placeholder	Pass
JavaScript	Pass; delegation placeholder	Pass
Java	Pass; delegation placeholder	Pass
C++	Pass; delegation placeholder	Pass

The C++ test results were reported as passing with GCC 14.2.0 and C++17. Keep the actual compiler output with the test evidence.

These examples demonstrate the structure of the Strategy pattern; they are not production-equivalent implementations of the MBTA application. In particular, the rate-limiter examples return a hard-coded `false` and do not test real rate-limiting behavior.

Source LOC, maximum function cyclomatic complexity, and median execution times were not measured. No estimates are reported.

5 LLM Records and Verification

Prompt files, model outputs, and LLM CSV records are maintained in the Part 1, Part 2, and Part 3 directories. These records should be interpreted alongside the source code and test evidence. Generated explanations alone do not prove that a pattern or test result is correct.

6 Findings

Two candidate Strategy collaborations were identified in the inspected MBTA source. The rate limiter delegates operations to configurable backends through a common behaviour. State parsing delegates to configured parser modules.

Two changes expanded rate-limiter test coverage, and one change improved parser exception handling. The cross-language examples illustrate the pattern structure but do not reproduce the full production behavior.

Team Contributions

- **Aadhav Gugan DU (AM.SC.U4CSE23201, Batch A):** Responsible for Part 1, including design pattern discovery, source code analysis, pattern identification, and documentation.
- **G Yaswanth Ram Saran (AM.SC.U4CSE23220, Batch A):** Responsible for Part 2, including code changes, refactoring, test development, test execution, and pattern evolution analysis.
- **KB Sai Prasad Reddy (AM.SC.U4CSE23227, Batch A):** Responsible for Part 3, including cross-language Strategy pattern implementations in Java, Python, JavaScript, and C++, test runners, and comparison documentation.

The team-level work covered pattern discovery, source evolution and testing, cross-language examples, documentation, and report preparation.

7 Limitations

The exact baseline Git commit could not be independently verified. Participant counts, source LOC, cyclomatic complexity, and median process timing were not measured. The cross-language examples are illustrative, and the targeted parser test used `-no-compile`. A separate compile/test attempt encountered a warning treated as an error.

These limitations are documented explicitly rather than replaced with estimated values.

8 References

- MBTA API source repository: `https://github.com/mbta/api`
- 23CSE455 Design Patterns case study instructions.
- Source files, diffs, CSV inventories, and test logs included in the Team A2 folder.